1.2 Introduction to Language Processors

Language processors comprises of assemblers, compilers and interpreters. Programmers write software's in a variety of languages. Compilers and interpreters translate these programming languages into instructions that are understood by the computer at machine level. Compilers and Interpreters are contained within the 'Systems services' category of software and they provide a facility for the execution of high level programs (programs such as C and BASIC).

Language processing activities occur due to difference between how software is made by the programmer and how it is implemented by the computer. The software programmer mentions the two domains i.e. application domain to present the ideas and execution domain for the carrying of these ideas.

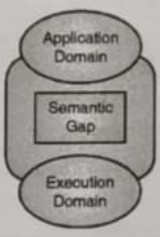


Fig. 1.2.1 : Semantic Gap between Application domain and Execution Domain

- The word semantics gives the set of rules used in different domains.
- Semantic Gap is the gap between the application and execution domains. Semantic gap takes a large development time, large efforts, and poor quality of software.
- **Programming Language** *(PL's)* domain is used to bridge the gap between the application domain and the execution domain.
  - The gap between the application and PL domain is given by Specification gap and is the space between the **two specifications** of the **same** task. This type of task is done by software development team.
  - The gap between the PL and execution domain is given by **Execution Gap.** It is the gap between the semantics of two programs written in different languages.
- Each domain has its own specification language to execute its tasks.

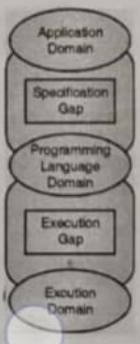


Fig. 1.2.2 : Various Programming Language Domain

Need of Language Domain

1. It helps in minimizing the errors caused by the semantic gap.
2. It helps in bridging the gap between the different domains and find out if there are ... input of any domain.

1.2.1 What are Language Processors ?

A Language Processor (LP) **is a program that performs tasks, such ... interpreting, required for processing a specified programming language.** ... the specification or execution gap. The program form input to a LP is *source program* and to its output as the *target program*.

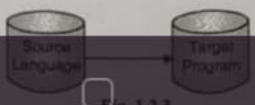


Fig. 1.2.3

1.2.2 Spectrum of Language Processors

execute its tasks.

Fig. 1.2.2 : Various Programming Language Domain

Need of Language Domain

1. It helps in minimizing the errors caused by the semantic gap.
2. It helps in bridging the gap between the different domains and find out if there are any errors in the input of any domain.

1.2.1 What are Language Processors ?

A Language Processor (LP) **is a program that performs tasks, such as translating and interpreting, required for processing a specified programming language.** It also bridges the specification or execution gap. The program form input to a LP is *source program* and to its output as the *target program*.

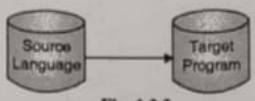


Fig. 1.2.3

1.2.2 Spectrum of Language Processors

- **System programming :** System programming aims to produce software which provides services to the computer hardware.

A spectrum of language processors is defined as :

- **Language Translator :** It bridges an execution gap between PL domain and execution domain e.g. Assembler, Compiler.
- **Detranslator :** Same as language translator but used in reverse direction i.e. from execution domain to PL domain.
- **Pre-processor :** It is also used to bridge the gap of the execution domain. Language processor converts a C++ program into a C program, hence it is a pre-processor.

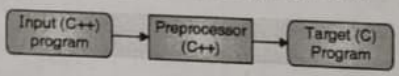


Fig. 1.2.4

- **Language Migrator :** It fills the specification gap between two PL's.

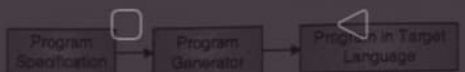
1.3 Language Processing Activities

The *Language Processing Activities* can be divided into those that bridge the specification gap and those that bridge the execution gap. There are two types of such activities :

- Program Generation Activities
- Program Execution Activities

1.3.1 Program Generation

It generates the target program in



E.g. MNEMONIC [DESTINATION], [SOURCE]

General format of mnemonic

MOV EAX, 1

↓ ↓ ↓

Mnemonic Destination Source

In this the value 1 is "moved" into the EAX operand (register).

2) Data Sections

They are instructions used to define data elements to hold data and variables. They define type of data, length and alignment of data.

3) Assembly Directives/Pseudo Operators

Assembly directives are instructions that are executed by an assembler at the assembly time. Symbolic assemblers allow programmers to associate arbitrary names (labels or symbols) with memory locations.

2.2.2 Assembly Language Program Contains Three Kinds of Statements

1) Imperative Statements

They are executable statements understood by the machines. It indicates an action to be performed during the execution of the assembled program. Each imperative statements typically translates into one machine instruction those are understood by the machine.

E.g. MOV A, 1

It states "Move" value one to register A

2) Declaration Statements

These are non executable statements. They declare constants or storage areas in a program.

E.g. X DS 2

(In this DS = Declare Storage. It reserves memory area of 1 word and associates the name X with it).

3) Assembler Directives

It instructs the assembler to perform certain actions during the assembly of a program.

E.g. ABC START 1000

END

(It indicates where to start assembly of the input program is performed).

2.2.3 An Assembly Language Program

When the user wants to communicate with the computer, he has a lot of languages :

Best for programmer

English

C/ C++/JAVA

:

:

Assembly language

Machine language

Best for machines

We will now discussed assembly language which is the most machine dependent language

	PROGRAM		COMMENTS
TEST	START		Identifies the name of the program
BEGIN	BALR	15, 0	Set register 15 to the address of the next instruction
	USING	BEGIN + 2, 15	Pseudo-opcode indicating to assembler register 15 is base register, and its content is address of next instruction
	SR	4, 4	Clear register 4
	L	3, ELEVEN	Load the number eleven into register 3
LOOP	L	2, DATA (6)	Load data into register 2
	A	2, FORTYSIX	Add 46 to register 2
	ST	2, DATA (6)	Store updated value of the data
	A	4, SIX	Add six to register four
	BCT	3, LOOP	Decrement register 3 by 1, if result non-zero branch

Module 2

CHAPTER 2

Assemblers

Syllabus Topics

Elements of Assembly Language programming, Assembly scheme, pass structure of assembler.

Assembler Design : Two pass assembler design and single pass Assembler, Design for Hypothetical / X86 family processor, data structures used.

2.1 Introduction

Assembly languages are a family of low-level languages for programming computers, micro controllers, microprocessors and other integrated circuits. They implement a symbolic representation of the numeric machine codes and other constants needed to program a particular CPU architecture. An assembly level language in thus specific to a certain physical or virtual computer architecture.

An assembler is a program used to convert the assembly level language statements into the target machine code. The assembler performs a one-to-one mapping from mnemonic instructions to machine statements and data. Mnemonic is a code usually from 1 to 5 letters that represents an op-code followed by one or more numbers. This is in contrast with high level languages in which single statement generally results in many machine instructions. An assembly language is a machine dependent, low level programming language which is specific to a particular computer system.

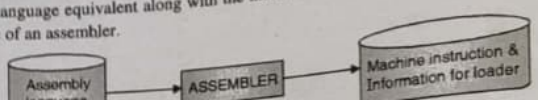
2.1.1 Advantages of Assembly Language

- It helps to get the better performance out of the processor as possible.
- It helps us to gain access to specific characteristics of the hardware that might not be possible from the higher level language.

2.1.2 Disadvantages of Assembly Language

- It always requires the use of an assembler to translate a source program into an object code.
- Assembly languages are specific to a given micro-processor/computer and hence are not portable.

- An **assembler** is a program that accepts an assembly language as input program and produces its machine language equivalent along with the information for the loader. Fig. 2.1.1 shows the basic schematic of an assembler.



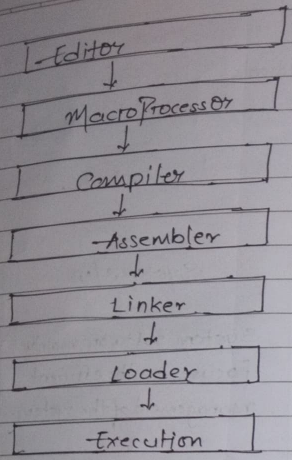
E.g.

Assignment No: 1

Q.1) Differentiate between Application software and System Software. Indicate the order in which following system programs are used from developing program upto its execution.
Assemblers, Loaders, Linkers, MacroProcessor, Compiler, Editors.

→

Parameter	System s/w	Application s/w
1. Definition	System software mainly focuses on the efficient management of the system.	An application program mainly focuses on solving a specific problem using computer as tool.
2. Purpose	System program supports the operation than rather any particular application.	The main focus is on the application not in the computer System.
3. Machine dependancy	System s/w is machine dependent.	Application s/w is Machine independent.
4. Program knowledge	system programmer required knowledge about internal computer architecture.	Application programmer required detail knowledge about high level language which is used to develop an operation.
5. Portability	system program becomes portable with concept of bootstrapping.	Application s/w becomes portable with the concept of cross computer.
6. Examples	OS, Compiler, Interpreter, Assembler, Loader, etc.	MS Access, Notepad Photoshops, etc.



Q.2] Define : System Programming.

- System Programming is the set of technologies techniques used to realize those design goals.
- system programming is the process of designing and implementary system level s/w such as operating system, compilers, Assemblers, device drivers and loaders that manage hardware resources and provide a platform for application programs.
- system Programming is the area of computer progra that focuses on developing system s/w which provi services to computer h/w and application prog
- It involves writing programs that controls, mana or interacts directly with the Computers h/w and O/.

• Key Characteristics of system Programming :

- Works close to the h/w.
- Emphasizes efficiency, performance and Reliability.
- Uses low-level or system-level languages (eg., C, C++, Assembly).
- Involves memory management, process management, I/O control.

• Examples of system Programs :

- Operating system (Linux, Windows, Kernel)
- compiler and Assemblers
- Device Drivers
- Linkers and Louders
- File systems.

Q.3] Compare between Compiler & Interpreter.

→

Parameter	Compiler	Interpreter
1. Definition	A Compiler translates the entire source program into machine code at once.	An Interpreter translates & executes the program line by line.
2. Translation Process	Whole program is Compiled before execution.	Each statement is trans- - lated and executed one at a time.

Parameter	Compiler	Interpreter
3. Execution Speed	Faster execution after Compilation.	Slower execution due repeated translation.
4. Output	Generates an object code executable file.	Does not generate obj code.
5. Error Detection	Reports all errors after compiling the entire program.	Stops at the first error encountered and report.
6. Memory requirement	Require more memory to store object code.	Requires less memory.
7. Debugging	Debugging is more difficult due to delayed error reporting.	Easier debugging since are shown line by line.
8. Portability	Executable is platform dependent.	Source code is portout interpreter exists.
9. Examples of language	C, C++, Java (uses both Compiler & interpreter)	Python, JavaScript, Ruby.
10 Use Case	Used where performed is Critical.	Use for Scripting.

Sundaram

FOR EDUCATIONAL USE

Q.4] What is Language Processor? Ex

→ - A language processor (LP) is
tasks, such as translating and
processing a specified program
- It also bridges the specifi
- The program form input
and to its output of the targ
- processors comprise
interpreters.

Proces

system

- Detran

- Pre processors
of the

Sundaram

Q5) Explain various databases used in 2 pass assembler design with suitable eg.

→ A two-pass assembler translates an assembly language program into machine code in 2 scans (passes) of the source program

- Pass 1 : Analyze the source program & builds various databases.
- Pass 2 : Uses these databases to generate final machine code.

↳ Database used in Two Pass Assembler

1. Symbol Table (SYMTAB)

- Stores information about symbols (labels) used
- Helps in resolving addresses of symbol during Pass 2.

Symbol	address	Length
LOOP	202	1
NEXT	214	1
LAST	216	1
A	217	1
BACK	202	1
B	218	1

2. Literal Table (LITAB)

- stores information about literals (constants with '=' sign)
- Literals are assigned addresses only when LTORG or END.
- Each entry in the LITAB pertains to a literal.
- It contains the fields literal and address.

Q.7] What is forward reference problem? How is it resolved in two pass assembler?

→ A forward reference occurs when a symbol (label) is used in an instruction before it is defined in the assembly language program.
- In such cases the assembler does not the address of the symbol at the time it is encountered.

↳ Resolution of forward reference in 2-pass assembler

- A two pass assembler resolves the forward reference problem by dividing the assembly process two passes.

→ Pass 1 : Define symbol Phase

- Scans the source program line by line.
- Assigns address to all symbols when they are defined.